

JSON Schemas

This document defines the JSON schemas used by skill-creator.

evals.json

Defines the evals for a skill. Located at `evals/evals.json` within the skill directory.

```json

```
{
 "skill_name": "example-skill",
 "evals": [
 {
 "id": 1,
 "prompt": "User's example prompt",
 "expected_output": "Description of expected result",
 "files": ["evals/files/sample1.pdf"],
 "expectations": [
 "The output includes X",
 "The skill used script Y"
]
 }
]
}
```

```

****Fields:****

- `skill\_name`: Name matching the skill's frontmatter
- `evals[].id`: Unique integer identifier
- `evals[].prompt`: The task to execute
- `evals[].expected\_output`: Human-readable description of success
- `evals[].files`: Optional list of input file paths (relative to skill root)
- `evals[].expectations`: List of verifiable statements

history.json

Tracks version progression in Improve mode. Located at workspace root.

```json

```
{
 "started_at": "2026-01-15T10:30:00Z",
 "skill_name": "pdf",
 "current_best": "v2",
 "iterations": [
 {
 "version": "v0",
 "parent": null,
 "expectation_pass_rate": 0.65,
 "grading_result": "baseline",
 "is_current_best": false
 },
],
}
```

```

 {
 "version": "v1",
 "parent": "v0",
 "expectation_pass_rate": 0.75,
 "grading_result": "won",
 "is_current_best": false
 },
 {
 "version": "v2",
 "parent": "v1",
 "expectation_pass_rate": 0.85,
 "grading_result": "won",
 "is_current_best": true
 }
]
}
...

Fields:
- `started_at`: ISO timestamp of when improvement started
- `skill_name`: Name of the skill being improved
- `current_best`: Version identifier of the best performer
- `iterations[].version`: Version identifier (v0, v1, ...)
- `iterations[].parent`: Parent version this was derived from
- `iterations[].expectation_pass_rate`: Pass rate from grading
- `iterations[].grading_result`: "baseline", "won", "lost", or "tie"
- `iterations[].is_current_best`: Whether this is the current best version

grading.json

Output from the grader agent. Located at `/grading.json`.

```json
{
  "expectations": [
    {
      "text": "The output includes the name 'John Smith'",
      "passed": true,
      "evidence": "Found in transcript Step 3: 'Extracted names: John Smith, Sarah Johnson'"
    },
    {
      "text": "The spreadsheet has a SUM formula in cell B10",
      "passed": false,
      "evidence": "No spreadsheet was created. The output was a text file."
    }
  ],
  "summary": {
    "passed": 2,
    "failed": 1,
    "total": 3,
    "pass_rate": 0.67
  },
  "execution_metrics": {

```

```

    "tool_calls": {
      "Read": 5,
      "Write": 2,
      "Bash": 8
    },
    "total_tool_calls": 15,
    "total_steps": 6,
    "errors_encountered": 0,
    "output_chars": 12450,
    "transcript_chars": 3200
  },
  "timing": {
    "executor_duration_seconds": 165.0,
    "grader_duration_seconds": 26.0,
    "total_duration_seconds": 191.0
  },
  "claims": [
    {
      "claim": "The form has 12 fillable fields",
      "type": "factual",
      "verified": true,
      "evidence": "Counted 12 fields in field_info.json"
    }
  ],
  "user_notes_summary": {
    "uncertainties": ["Used 2023 data, may be stale"],
    "needs_review": [],
    "workarounds": ["Fell back to text overlay for non-fillable fields"]
  },
  "eval_feedback": {
    "suggestions": [
      {
        "assertion": "The output includes the name 'John Smith'",
        "reason": "A hallucinated document that mentions the name would also pass"
      }
    ],
    "overall": "Assertions check presence but not correctness."
  }
}
...

**Fields:**
- `expectations[]`: Graded expectations with evidence
- `summary`: Aggregate pass/fail counts
- `execution_metrics`: Tool usage and output size (from executor's metrics.json)
- `timing`: Wall clock timing (from timing.json)
- `claims`: Extracted and verified claims from the output
- `user_notes_summary`: Issues flagged by the executor
- `eval_feedback`: (optional) Improvement suggestions for the evals, only present when the grader identifies issues worth raising
---

## metrics.json

Output from the executor agent. Located at `/outputs/metrics.json`.

```

```

```json
{
 "tool_calls": {
 "Read": 5,
 "Write": 2,
 "Bash": 8,
 "Edit": 1,
 "Glob": 2,
 "Grep": 0
 },
 "total_tool_calls": 18,
 "total_steps": 6,
 "files_created": ["filled_form.pdf", "field_values.json"],
 "errors_encountered": 0,
 "output_chars": 12450,
 "transcript_chars": 3200
}
```

```

****Fields:****

```

- `tool_calls`: Count per tool type
- `total_tool_calls`: Sum of all tool calls
- `total_steps`: Number of major execution steps
- `files_created`: List of output files created
- `errors_encountered`: Number of errors during execution
- `output_chars`: Total character count of output files
- `transcript_chars`: Character count of transcript
---
```

timing.json

Wall clock timing for a run. Located at ``<run-dir>/timing.json``.

****How to capture:**** When a subagent task completes, the task notification includes ``total_tokens`` and ``duration_ms``. Save these immediately – they are not persisted anywhere else and cannot be recovered after the fact.

```

```json
{
 "total_tokens": 84852,
 "duration_ms": 23332,
 "total_duration_seconds": 23.3,
 "executor_start": "2026-01-15T10:30:00Z",
 "executor_end": "2026-01-15T10:32:45Z",
 "executor_duration_seconds": 165.0,
 "grader_start": "2026-01-15T10:32:46Z",
 "grader_end": "2026-01-15T10:33:12Z",
 "grader_duration_seconds": 26.0
}
```

```

benchmark.json

Output from Benchmark mode. Located at `benchmarks/<timestamp>/benchmark.json`.

```
```json
{
 "metadata": {
 "skill_name": "pdf",
 "skill_path": "/path/to/pdf",
 "executor_model": "claude-sonnet-4-20250514",
 "analyzer_model": "most-capable-model",
 "timestamp": "2026-01-15T10:30:00Z",
 "evals_run": [1, 2, 3],
 "runs_per_configuration": 3
 },

 "runs": [
 {
 "eval_id": 1,
 "eval_name": "Ocean",
 "configuration": "with_skill",
 "run_number": 1,
 "result": {
 "pass_rate": 0.85,
 "passed": 6,
 "failed": 1,
 "total": 7,
 "time_seconds": 42.5,
 "tokens": 3800,
 "tool_calls": 18,
 "errors": 0
 },
 "expectations": [
 {"text": "...", "passed": true, "evidence": "..."}
],
 "notes": [
 "Used 2023 data, may be stale",
 "Fell back to text overlay for non-fillable fields"
]
 }
],

 "run_summary": {
 "with_skill": {
 "pass_rate": {"mean": 0.85, "stddev": 0.05, "min": 0.80, "max": 0.90},
 "time_seconds": {"mean": 45.0, "stddev": 12.0, "min": 32.0, "max": 58.0},
 "tokens": {"mean": 3800, "stddev": 400, "min": 3200, "max": 4100}
 },
 "without_skill": {
 "pass_rate": {"mean": 0.35, "stddev": 0.08, "min": 0.28, "max": 0.45},
 "time_seconds": {"mean": 32.0, "stddev": 8.0, "min": 24.0, "max": 42.0},
 "tokens": {"mean": 2100, "stddev": 300, "min": 1800, "max": 2500}
 },
 "delta": {
 "pass_rate": "+0.50",
 "time_seconds": "+13.0",

```

```

 "tokens": "+1700"
 }
},

"notes": [
 "Assertion 'Output is a PDF file' passes 100% in both configurations - may not
differentiate skill value",
 "Eval 3 shows high variance (50% ± 40%) - may be flaky or model-dependent",
 "Without-skill runs consistently fail on table extraction expectations",
 "Skill adds 13s average execution time but improves pass rate by 50%"
]
}
```

**Fields:**
- `metadata`: Information about the benchmark run
- `skill_name`: Name of the skill
- `timestamp`: When the benchmark was run
- `evals_run`: List of eval names or IDs
- `runs_per_configuration`: Number of runs per config (e.g. 3)
- `runs[]`: Individual run results
    - `eval_id`: Numeric eval identifier
    - `eval_name`: Human-readable eval name (used as section header in the viewer)
    - `configuration`: Must be `"with_skill"` or `"without_skill"` (the viewer uses this
exact string for grouping and color coding)
    - `run_number`: Integer run number (1, 2, 3...)
    - `result`: Nested object with `pass_rate`, `passed`, `total`, `time_seconds`,
`tokens`, `errors`
- `run_summary`: Statistical aggregates per configuration
    - `with_skill` / `without_skill`: Each contains `pass_rate`, `time_seconds`, `tokens`
objects with `mean` and `stddev` fields
    - `delta`: Difference strings like `"+0.50"`, `"+13.0"`, `"+1700"`
- `notes`: Freeform observations from the analyzer

**Important:** The viewer reads these field names exactly. Using `config` instead of
`configuration`, or putting `pass_rate` at the top level of a run instead of nested
under `result`, will cause the viewer to show empty/zero values. Always reference this
schema when generating benchmark.json manually.

---

## comparison.json

Output from blind comparator. Located at `/comparison-N.json`.

```json
{
 "winner": "A",
 "reasoning": "Output A provides a complete solution with proper formatting and all
required fields. Output B is missing the date field and has formatting
inconsistencies.",
 "rubric": {
 "A": {
 "content": {
 "correctness": 5,
 "completeness": 5,

```

```

 "accuracy": 4
 },
 "structure": {
 "organization": 4,
 "formatting": 5,
 "usability": 4
 },
 "content_score": 4.7,
 "structure_score": 4.3,
 "overall_score": 9.0
},
"B": {
 "content": {
 "correctness": 3,
 "completeness": 2,
 "accuracy": 3
 },
 "structure": {
 "organization": 3,
 "formatting": 2,
 "usability": 3
 },
 "content_score": 2.7,
 "structure_score": 2.7,
 "overall_score": 5.4
}
},
"output_quality": {
 "A": {
 "score": 9,
 "strengths": ["Complete solution", "Well-formatted", "All fields present"],
 "weaknesses": ["Minor style inconsistency in header"]
 },
 "B": {
 "score": 5,
 "strengths": ["Readable output", "Correct basic structure"],
 "weaknesses": ["Missing date field", "Formatting inconsistencies", "Partial data
extraction"]
 }
},
"expectation_results": {
 "A": {
 "passed": 4,
 "total": 5,
 "pass_rate": 0.80,
 "details": [
 {"text": "Output includes name", "passed": true}
]
 },
 "B": {
 "passed": 3,
 "total": 5,
 "pass_rate": 0.60,
 "details": [
 {"text": "Output includes name", "passed": true}
]
 }
}

```

```

]
 }
}
}
...

analysis.json

Output from post-hoc analyzer. Located at `<grading-dir>/analysis.json`.

```json
{
  "comparison_summary": {
    "winner": "A",
    "winner_skill": "path/to/winner/skill",
    "loser_skill": "path/to/loser/skill",
    "comparator_reasoning": "Brief summary of why comparator chose winner"
  },
  "winner_strengths": [
    "Clear step-by-step instructions for handling multi-page documents",
    "Included validation script that caught formatting errors"
  ],
  "loser_weaknesses": [
    "Vague instruction 'process the document appropriately' led to inconsistent behavior",
    "No script for validation, agent had to improvise"
  ],
  "instruction_following": {
    "winner": {
      "score": 9,
      "issues": ["Minor: skipped optional logging step"]
    },
    "loser": {
      "score": 6,
      "issues": [
        "Did not use the skill's formatting template",
        "Invented own approach instead of following step 3"
      ]
    }
  },
  "improvement_suggestions": [
    {
      "priority": "high",
      "category": "instructions",
      "suggestion": "Replace 'process the document appropriately' with explicit steps",
      "expected_impact": "Would eliminate ambiguity that caused inconsistent behavior"
    }
  ],
  "transcript_insights": {
    "winner_execution_pattern": "Read skill -> Followed 5-step process -> Used validation script",
    "loser_execution_pattern": "Read skill -> Unclear on approach -> Tried 3 different methods"
  }
}

```



```
}  
}  
...
```